



TCC: Geração Automática de Código LaTeX a Partir de Expressões Matemáticas Manuscritas

Autor: Emanuel Henrique Farias

Bruno Henrique Duarte
João Vítor Paiva Nunes Pereira
Departamento de Ciência da Computação
Professor André Tavares da Silva

O Problema

“Escrever expressões matemáticas complexas em um computador usando editores de textos é uma tarefa trabalhosa”.

A matemática permeia todas as ciências e possui um vasto conjunto de símbolos (aproximadamente 2 mil) utilizados para expressar os mais diferentes conceitos. Tais símbolos não estão presentes em teclados e, na maioria dos editores, são escritos através de comandos.

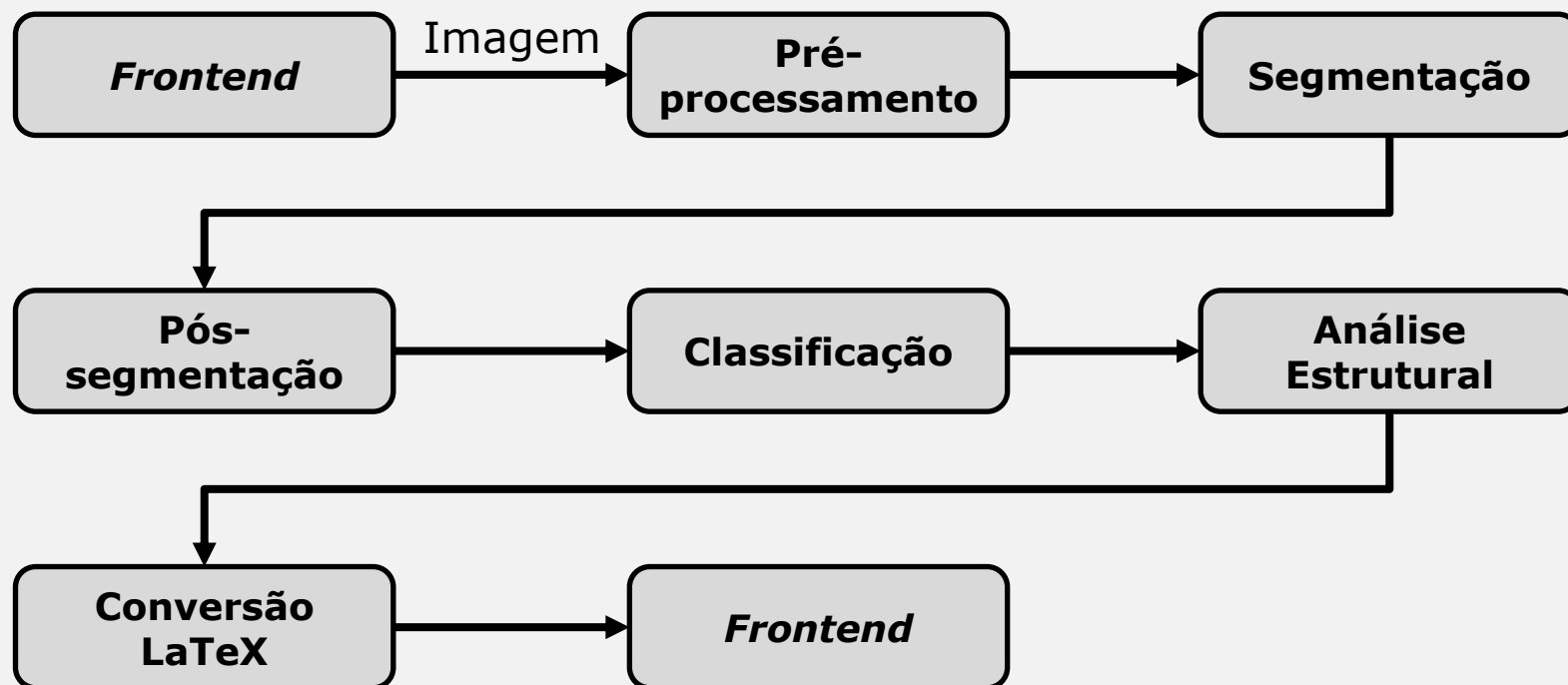
O Problema

“Desenvolver um reconhecedor automatizado de expressões matemáticas, além de facilitar a entrada de expressões em um editor de textos, teria outras utilidades como a conversão de livros e artigo científicos para o formato digital, ajudar na entrada de dados em softwares especializados em manipulação algébrica, facilitar a criação de documentos LaTeX sem precisar conhecer os comandos, e até mesmo auxiliar portadores de deficiência visual pela leitura de documentos e papeis com expressões matemáticas”

Objetivo

“O objetivo deste trabalho é desenvolver uma aplicação web em que o usuário possa fazer upload de uma imagem (que contém uma expressão matemática manuscrita) e o sistema deverá interpretar essa imagem e devolver para o usuário o código LaTeX da expressão, e ainda, o navegador deverá renderizá-la para que a pessoa possa visualizar”.

Etapas da Aplicação



Adaptado do trabalho.

Técnicas utilizadas

- **Pré-processamento e Processamento de Imagens:** Conversão para tons de cinza, filtro de mediana, limiarização adaptativa (*Adaptive Mean Thresholding*), segmentação por componentes conexos (*findContours*), operações morfológicas de dilatação e redimensionamento de imagem;
- **Classificação:** Redes Neurais Convolucionais (CNN), funções de ativação ReLU e *softmax*, função de erro *cross-entropy* (log loss), algoritmo otimizador ADAM, *Stochastic Gradient Descent* (SGD), *dropout*, *backpropagation* e *cross-validation*;
- **Análise Estrutural:** Grafos, Árvore Geradora Mínima, Árvore de Baselines (BST - Baseline Structure Tree) e técnica de detecção de colisão de círculos para cálculo de pesos.

Recursos utilizados

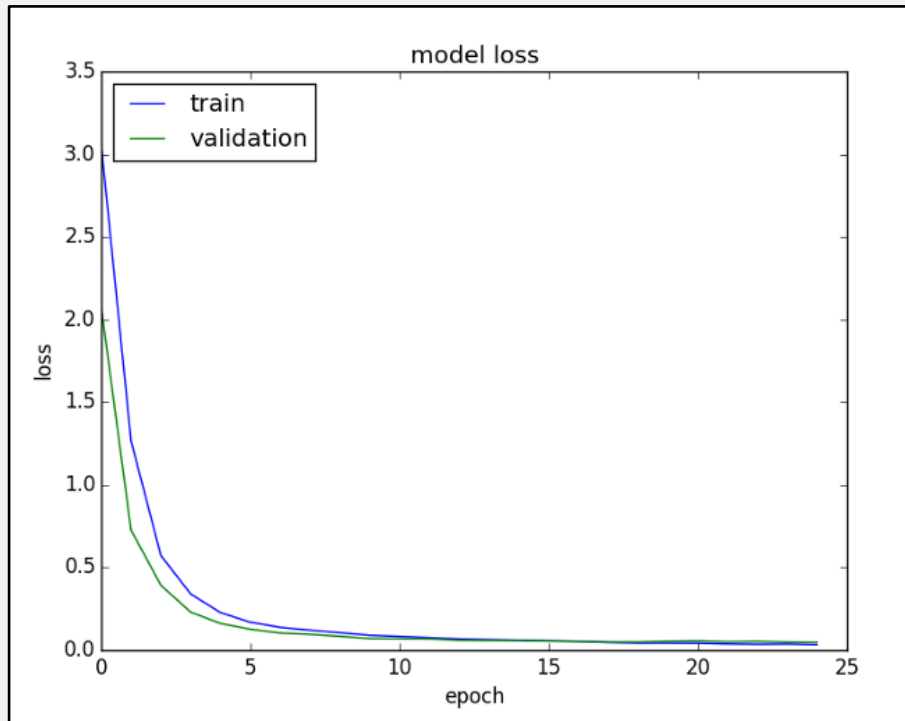
- **Aplicação WEB:** *framework* Sinatra da linguagem Ruby;
 - **Seleção da Imagem:** biblioteca Cropper da linguagem JavaScript;
 - **Renderização da Expressão LaTeX:** biblioteca MathJax da linguagem JavaScript.
- **Sistema Reconhecedor:**
 - **Pré-processamento, Segmentação e Pós-segmentação:** biblioteca OpenCV da linguagem C++;
 - **Treinamento e Classificação:** bibliotecas Keras e TensorFlow da linguagem Python e os *datasets* HWRT, Chars74K, MNIST e um criado pelo próprio autor;
 - **Análise Estrutural:** linguagem Ruby;
- **Intel Core i7 4790K no treinamento da CNN.**

O que foi feito?

O autor implementou todas as etapas necessárias para um reconhecedor *offline* de expressões matemáticas. Ele desenvolveu uma aplicação web com uma ferramenta de seleção de imagens e um sistema de reconhecimento para a montagem da árvore da expressão e a conversão final para uma *string* de código LaTeX.

Vale destacar que a etapa de classificação exigiu um treinamento da CNN utilizada. Foram utilizados os quatro *datasets* citados para o treinamento (66271 símbolos foram usado – 60% para treinamento, 20% para validação e 20% para testes).

Resultados Obtidos



Retirado do trabalho.

“O classificador obteve uma taxa média de erro de 1.407% em três execuções de treinamento, e o desvio-padrão foi de ± 0.061 . Cada execução do treinamento demorou aproximadamente 27 minutos [...]”.

Resultados Obtidos

As expressões matemáticas que o sistema pode reconhecer são:

- Aritméticas básicas (soma, subtração, multiplicação e divisão);
- Polinômios;
- Funções Racionais (divisão de polinômios);
- Funções Exponenciais;
- Somatórios;
- Integrais.

Expressões matemáticas que o sistema não reconhece corretamente:

- Funções Trigonométricas;
- Números com casas decimais (com vírgulas e pontos);
- Raízes.

Texto do autor.

Resultados Obtidos

Restrições:

- Apenas uma expressão por imagem;
- Folha de escrita sem pauta;
- A expressão deve ser escrita em uma única linha;
- Não pode haver sobreposição de símbolos;
- A expressão deve ser alinhada horizontalmente.

Resultados Obtidos

Problemas:

- Pré-processamento pode deformar demais a imagem e afetar a segmentação e o restante do processo;
- Segmentação não opera direito com símbolos justapostos;
- Pós-segmentação pode redimensionar demais um símbolo e afetar a sua classificação (o que exige mais treinamento da rede neural);
- A classificação depende totalmente da base de dados utilizada;
- A análise estrutural apresentou diversos problemas na identificação de sobrescritos, subscritos e subexpressões.

■ Conclusão do trabalho

“Este foi um trabalho completo com a implementação de todas as etapas de um reconhecedor offline de expressões matemáticas [(algo raro na literatura pesquisada)]. O trabalho apresentou bons resultados, mesmo utilizando métodos comuns na área de processamento de imagens. Para implementar um sistema mais robusto, todas as etapas podem ser melhoradas com o uso de métodos mais avançados [...]”.

■ Áreas Relacionadas

- **Processamento de Imagens e Visão Computacional:**
porque o sistema requer a aquisição de uma imagem digital e a aplicação de filtros matemáticos, binarização e segmentação de pixels para isolar os componentes (símbolos) do fundo do papel;
- **Aprendizado de Máquina e Reconhecimento de Padrões:** porque o projeto utiliza CNNs que são treinadas com bases de dados para extrair características e prever de forma automática a qual classe uma determinada imagem pertence.

■ Áreas Relacionadas

- **Estruturas de Dados e Algoritmos:** porque o trabalho utiliza conceitos como grafos, árvores geradoras mínimas e algoritmos de busca e ordenação para as diversas etapas da aplicação.
- **Desenvolvimento Web:** porque englobou o desenvolvimento prático de uma aplicação acessível via navegador, exigindo a integração de programas construídos em diferentes linguagens (C++, Python, Ruby) e criação de interfaces *frontend* e *backend*.

Referências Bibliográficas

- FARIAS, Emanuel Henrique. **Geração automática de código LaTeX a partir de expressões matemáticas manuscritas.** 2016. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) – Universidade do Estado de Santa Catarina, Joinville, 2016.